ragsec — 프로젝트 요약 보고서

한국 금융권 RAG 에이전트의 간접 프롬프트 주입(IPI)에 의한 개인정보 유출과 다단계 가드레일의 효과 측정 테스트베드

작성 목적: 지금까지 진행한 작업의 구조 · 흐름 · 접근 방식을 한눈에 정리 · 2026-10-29

---

0. 한 줄 요약

“회사 문서를 찾아 답하는 AI 비서(RAG)가, 악성 지시가 숨겨진 문서에 속아 고객의 민감정보를 유출하는지”를 안전하게(모두 가짜 데이터로, 외부 전송 차단) 반복 실험하고, 여러 겹의 방어막(가드레일)이 그것을 얼마나 막는지 객관적 지표로 측정하기 위한 연구용 소프트웨어다.

1. 프로젝트 개요

목표 및 주요 내용

은행 등 금융기관이 내부 문서를 검색해 답하는 LLM 비서(RAG, Retrieval-Augmented Generation)를 도입할 때, 검색되는 문서 안에 공격자가 심어둔 지시문(간접 프롬프트 주입, IPI)이 섞여 있으면 모델이 그 지시를 따라 고객의 주민등록번호 · 계좌 · 카드번호 같은 개인정보를 외부로 빼돌릴 수 있다. 이 프로젝트는 그 위험을 실제 공격 없이, 합성 데이터로 재현하고, 정규식 PII 탐지 · 한/영 IPI 규칙 · Llama Guard · 출처 신뢰도 · 응답 유출 검사 등 여러 방어막을 끄고/켜 가며 공격 성공률 · 오탐율 · 지연시간을 측정하는 재현 가능한 실험 체계를 만든다.

사용 기술 · 도구 · 하드웨어

구분	사용 내역
언어/런타임	Python 3.12 (uv로 의존성 고정), 약 6,000줄 / 37개 모듈
LLM 서빙	Ollama (로컬) — exaone3.5:2.4b(생성), bge-m3(임베딩), llama-guard3:1b(안전분류)
백엔드/UI	FastAPI(API) + Streamlit(실험 콘솔), ChromaDB 1.5.9(벡터DB)
하드웨어	노트북 1대: RTX 4050 Laptop(6GB VRAM), 16GB RAM — 단일 GPU
검증	pytest 테스트 151개 통과(모의 백엔드, GPU 불필요) + 실제 Ollama 스모크 확인

※ 모든 데이터는 합성이며, 식별번호는 형식만 맞고 체크섬이 틀린 ‘카나리(추적용 미끼)’다. 외부 전송은 로컬 '모의 싱크(mock sink)'로만 시뮬레이션하며 실제 네트워크 전송은 코드 차원에서 차단된다.

2. 시스템 구조와 핵심 흐름

2.1 전체 처리 흐름 (요청 1건 기준)

단계	하는 일
1) 사용자 인증	요청한 user_id로 역할 · 소속(tenant)을 서버 정책에서 확인
2) 질문 임베딩	bge-m3로 질문을 벡터로 변환
3) 권한 기반 검색	소속 · 역할 · 정보등급 · 유효기간 필터를 걸어 ChromaDB에서 관련 문서만 검색(권한 없는 문서는 애초에
4) 수집 단계 격리	수집 시 위험 표시된 조각을 검색 결과에서 제외(가드 모드에 따라)
5) 생성 前 검사(PRE)	질문 · 검색된 조각에 대해 PII/IPI/Llama Guard/출처 검사 → 허용 · 마스킹 · 격리 · 차단 결정
6) 프롬프트 조립 + 생성	허용된 근거 문서 + 질문으로 LLM이 답변 생성(스트리밍 금지: 검사 전 노출 방지)

7) 생성 後 검사(POST)	응답 · 도구 동작에 대해 카나리 유출 · PII 검사 → 차단/마스킹
8) 모의 도구 실행	mock - sink:// · internal:// 목적지만 로컬 실행, 그 외 ‘ 시도 ’ 로만 기록(전송 안 함)
9) 유출 측정 + 기록	카나리 일치 · 권한외 공개 등을 측정해 JSONL 로그로 저장(원문 PII · 전체 프롬프트는 저장 안 함)

핵심 설계 원칙: ‘ 탐지(detector) ’ 와 ‘ 집행(enforcer) ’ 을 분리했다. 각 가드 플러그인은 ‘ 이건 위험해 보임 ’ 이라고 추천만 하고, 하나의 집행기가 정책과 합쳐 최종 결정(허용/마스킹/격리/차단)을 내린다. 덕분에 ‘ 탐지 성능 ’ 과 ‘ 정책 효과 ’ 를 따로 측정할 수 있다.

2.2 가드 모드(방어막 조합)

OFF(없음) · REGEX_ONLY(정규식 PII) · LLAMA_GUARD(안전분류) · HYBRID(전부) · HYBRID_NO_IPI/IPI_ONLY(기여도 분해용). OFF에서도 인증 · 권한검색 · 업로드제한 · 로컬경계는 항상 켜져 있다. 같은 사건(case)에 대해 모드만 바꿔 쌍대(paired) 비교한다.

2.3 모듈 지도(어디에 무엇이 있나)

영역	모듈	역할
설정/스키마	`config, schemas, errors`	VRAM 프로파일 · 루프백 검증 / Pydantic 계약 / 예외 체계
문서 처리	`parsing, textproc, ingestion`	PDF · TXT 파싱 / 유니코드 정규화 / 청크 · 임베딩 · 버전 인덱싱
저장/검색	`store(ChromaDB)`	버전별 인덱스 · 권한 필터(fail - closed) · 스테이징/롤백
모델 서빙	`llm/ollama·admin·mock, scheduler`	런타임 추론 클라이언트(엔드포인트 허용목록) / 진단 분리 / 단일 GPU 스케줄
가드레일	`guards/pii·ipi·llama_guard·runner`	PII/IPI 규칙 · 안전분류 · 출처 / 탐지→집행 분리
오케스트레이션	`rag`	위 2.1 흐름 전체를 엮는 핵심
평가/지표	`evaluation, metrics, sink`	카나리 유출 측정 / ASR · FPR · F1 집계 / 로컬 모의 싱크
실험/운영	`experiments, pilot, api, cli, datasets`	실험 러너 · 프리즈 / API · CLI / 합성 데이터 생성

3. 어떤 문제를, 어떻게 접근했나

문제 A — ‘PII가 있다 = 공격/유출’로 뭉뚱그리면 안 된다

접근: (1)문서에 PII가 있는가, (2)사용자가 볼 권한이 있는가, (3)문서에 악성 지시가 있는가, (4)모델이 그 지시를 따랐는가, (5)권한 없이 공개됐는가 — 다섯 가지를 코드와 지표에서 끝까지 분리. ‘권한 있는 담당자가 자기 답변에서 본인 업무상 PII를 본 것’은 유출로 세지 않는다.

문제 B — 접근통제(검색 권한)와 가드레일을 섞으면 원인을 못 가린다

접근: 권한 필터는 검색 ‘전에’ 항상 적용(기본). 일부러 약화한 조건은 별도 라벨. ‘권한 때문에 애초에 안 보인 것’과 ‘IPI를 따라 유출한 것’을 로그로 구분.

문제 C — 단일 6GB GPU에서 3개 모델을 동시에 못 올린다

접근: VRAM 프로파일(4/6/8GB)로 동거 가능한 모델 조합을 계획, 단일 스케줄러(프로세스락+파일락)로 GPU 작업 직렬화, 임베딩은 CPU로 오프로드, 콜드스타트와 정상상태 지연을 분리 측정.

문제 D — 실제 공격/실제 개인정보를 쓰면 위험하다

접근: 전부 합성 데이터 + 체크섬이 틀린 카나리, 외부 전송은 로컬 모의 싱크로만, Ollama는 루프백 전용 + 엔드포인트 허용목록으로 공격면 축소(이는 CVE 패치가 아니라 애플리케이션 수준 방어임을 명시).

문제 E — 소프트웨어 테스트 통과를 ‘연구 성과’로 착각하면 안 된다

접근: 공격 payload가 아직 placeholder라 공격 효과(ASR 등)는 ‘측정 불가(N/A)’로 정직하게 표기. 사람 검토를 거친 공격 사례가 있어야만 효능 실험이 돌도록 ‘검토 게이트’와 ‘콘텐츠 프리즈’로 막아둠.

4. 캡스톤/학부 졸업 작품 수준 평가

종합 의견

엔지니어링 · 연구설계 측면은 학부 졸업작품 수준을 분명히 넘어서며, 석사 초기 연구에 근접한다. 다만 ‘실험 결과(측정치)’가 아직 없다는 점에서, 지금 상태로 ‘졸업 논문’을 제출하려면 한 번의 실제 실험이 반드시 필요하다. 즉, ‘틀(framework)’은 매우 우수하고 ‘결과(findings)’가 비어 있는 상태다.

강점 (졸업작품 기준 상위권 요소)

- 명확한 연구질문(RQ1~RQ5)과 실험 설계: 쌍대 비교 · 순서 균형 · 신뢰구간 · McNemar · 효과크기까지 지표가 엄밀.
- 보안 · 재현성 수준이 높음: 루프백 전용 · 엔드포인트 허용목록 · HMAC는 제거했으나 가드/권한 경계 · 콘텐츠 프리즈 · 해시 검증 유지.
- 연구 타당성 통제: placeholder 제외(N/A), 공격 성과를 ‘지시 수행/제안/전송시도/실제 수신/최종 공개’로 분리.
- 약 6,000줄 · 37모듈 · 테스트 151개 통과 + 실제 모델 스모크 검증. 문서(DESIGN/REVIEW_PACKAGE 등)도 충실.

아쉬운 점 → 보완하면 완성도가 급상승하는 요소

- (가장 중요) 실제 공격 실험 1회: 보안연구자가 검토한 IPI payload 2~3개로 DEV 파일럿을 돌려 OFF 대비 HYBRID의 ASR/FPR/지연을 표 · 그래프로 제시 → ‘결과가 있는 논문’이 됨.

- 지표 시각화: 모드별 ASR · FPR · F1 막대그래프, 지연 p50/p95, 방어 트레이드오프 곡선(보호↔지연↔유용성).
- egress 정책 2차 요인화: 현재 ALLOWLIST에선 싱크 수신이 구조적으로 불가 → OPEN egress 별도 조건으로 실제 수신까지 관찰하는 실험 추가.
- 한국어 효능 측정(RQ4): 한/영/혼합/유니코드 변형별 탐지 재현율 비교(가설이 아닌 수치로).
- 검색 품질 튜닝: 교차문서 공격이 성립하려면 보호문서가 함께 검색돼야 함 → top-k · 임베딩 점검.
- 사람 평가/보조 평가(선택): 유용성(utility)을 키워드+소수 인적 평가 또는 Gemini 보조판정으로 보강.

요약 평가: 틀 9/10, 결과 1/10(미수행). 실제 파일럿 결과 + 그래프 3~4장만 추가하면 학부 우수 졸업작품/소논문으로 충분하다.

5. 코드 · 기술 쉬운 설명 (학부생/비전공자 눈높이)

5.1 큰 그림 비유

이 시스템은 ‘ 민원실 직원(AI) + 보안검색대 ’ 라고 보면 쉽다. 민원인이 질문하면 직원은 서류 캐비닛(벡터DB)에서 관련 서류를 꺼내 읽고 답한다. 그런데 어떤 서류에는 ‘ 이 고객 정보를 전부 이 주소로 보내라 ’ 같은 몰래 적힌 쪽지(악성 지시)가 들어 있을 수 있다. 그래서 서류를 꺼낼 때(권한 검색), 답하기 전(PRE 검사), 답한 뒤(POST 검사) 세 번의 보안검색대를 두고, 수상하면 가리거나(마스킹) 빼거나(격리) 막는다(차단). 바깥으로 뭔가 ‘ 보내는 ’ 행동은 진짜 우체통이 아니라 가짜 우체통(모의 싱크)으로만 가게 해 실제 유출을 원천 차단한다.

5.2 모듈별 역할 (왜 필요한가)

- rag.py (오케스트레이터)

민원실 직원 본인. 인증→검색→검사→생성→검사→기록의 전 과정을 지휘한다. 없으면 각 부품이 따로 논다.

- store.py (ChromaDB + 권한필터)

서류 캐비닛 + 사서. 내 소속 · 등급에 맞는 서류만 꺼내 준다. 권한 필터가 없으면(fail - closed) 아예 검색을 거부.

- guards/*.py (가드레일)

보안검색대. 정규식(PII 숫자 패턴), IPI 규칙(‘ 이전 지시 무시 ’ 같은 문구), Llama Guard(유해성 분류), 출처 신뢰도, 응답 유출검사. 각자 ‘ 의심 ’ 만 보고하고 runner가 최종 판정.

- llm/ollama.py (추론 클라이언트)

AI 모델에게 말 거는 창구. ‘ 정해진 3개 통로(chat/generate/embed) ’ 로만, 루프백으로만 말하게 제한해 오 · 남용(SSRF 등)을 줄인다. 진단용 통로(ps/tags)는 별도 관리자 클라이언트로 분리.

- scheduler.py (GPU 스케줄러)

작업 순번표. 6GB GPU에 모델이 몰리지 않게 한 번에 하나씩, 호환 안 되는 모델은 내리고 올린다.

- evaluation.py / metrics.py

채점관. 미끼(카나리)가 답변 · 전송에 나타났는지 세고, 공격성공률(ASR) · 오탐율(FPR) · F1을 계산. ‘ 권한 있는 공개 ’ 는 유출로 안 센다.

- sink.py (모의 싱크)

가짜 우체통. [[ACTION:send_report|to=...]] 형식의 ‘ 보내기 ’ 동작을 흉내만 내고 로컬 파일에 기록. 외부 주소는 ‘ 시도 ’ 로만 남기고 실제로 안 보냄.

- dataset_gen.py / experiments/pilot

시험지 출제 · 채점 운영. 합성 문서 · 사건을 만들고, 사람이 검토한 공격만 실험이 돌도록 게이트 · 프리즈(해시 고정)로 통제.

5.3 ‘ 간접 프롬프트 주입(IPI) ’ 한 문장

내가 AI에게 직접 나쁜 말을 하는 게 아니라, AI가 참고하려고 읽는 문서 속에 나쁜 지시를 숨겨 두어 AI가 그걸 진짜 명령으로 착각하게 만드는 공격이다. 이 프로젝트는 그 공격을 안전하게 재현하고 막는 실험대다.

부록: 현재 상태 — 소프트웨어 · 테스트(151개) · 안전장치는 완성. 공격 payload는 사람 검토 대기(placeholder)라 ‘ 공격 효과 수치 ’ 는 아직 없음(N/A). 다음 단계는 검토된 공격 2~3건으로 DEV 파일럿 1회 실행.